



KNOWLEDGE SHARE DOCUMENT

SchedulerAI

Tech Deep-Dive · Demo Prep · Q&A Reference

React 18 + Vite

Node.js + Express

MongoDB + Mongoose

GROQ / OpenAI / Claude / Gemini

Google Calendar + Outlook

Zoom + Teams + Google Meet

10s

TO SCHEDULE A
MEETING

4+

AI PROVIDERS
SUPPORTED

130h

SAVED PER YEAR /
USER



June 2025



Biswajit Singha



v1.0 — Manager Demo

Table of Contents

1

Product Overview

4 questions

2

Tech Stack — Why We Chose Each Tool

10 questions

3

AI Architecture & Pluggability

7 questions

4

Conversational Chat Flow

6 questions

5

Calendar & Conflict Detection

5 questions

6

Meeting Platform Integrations

6 questions

7

Authentication & Security

5 questions

8

System Architecture

5 questions

9

Business Value & Demo Script

3 topics

10

Limitations & Honest Answers

4 questions

11

Future Roadmap

3 questions

12

Pre-Demo Checklist

Run guide



Section 1 — Product Overview

What SchedulerAI is, who it's for, and why it exists

Q What does SchedulerAI do in one sentence?

SchedulerAI is an **AI-powered meeting scheduler** that lets users describe a meeting in plain English — and the app automatically finds a free time, creates the calendar event, and sends invites with a video meeting link, without any manual steps.

Q Who is the target user?

Professionals who schedule 5+ meetings a week and waste time on manual calendar juggling, finding availability, copying Zoom links, and writing invite emails. PMs, managers, and executives are the primary users.

Q What problem does it solve? What is the real pain?

Scheduling a single meeting requires: checking your calendar, finding a free slot, cross-checking attendees' availability, creating the calendar event, generating a Zoom/Teams link, pasting it into the invite, writing the email, and sending it. That's **5–10 minutes of manual work per meeting**.

SchedulerAI collapses this to a **single plain-English sentence**. The AI handles extraction, calendar checking, conflict resolution, meeting link creation, and invite sending — in one automated flow.



What makes it different from Calendly or Google Calendar's AI?

FEATURE	SCHEDULERAi	CALENDLY	GOOGLE CALENDAR AI
Natural language input	✓ Full NLP	✗	~ Limited
Proactive scheduling (you initiate)	✓	✗ External only	✓
Multi-calendar conflict check	✓ Google + Outlook	~ Single cal	✗
Multi-platform meetings	✓ Zoom/Teams/Meet	~ Meet only	~ Meet only
Conversational rescheduling	✓	✗	✗
Pluggable AI backend	✓ 4 providers	✗	✗
Admin provider control	✓	✗	✗



Section 2 — Tech Stack: Why We Chose Each Tool

Reasoned decisions behind every technology choice

Q Why React 18 for the frontend? Why not Vue, Angular, or Svelte?

React was chosen for three concrete reasons:

- **Ecosystem maturity** — libraries for OAuth redirects, toast notifications, and routing are battle-tested and well-documented in React.
- **useRef for stale closure control** — the chat flow has async callbacks (platform selection, email confirmation) that need to read live state without triggering re-renders. React's `useRef` solves this precisely. Vue's reactivity system makes this harder to control explicitly.
- **Component reuse** — `PlatformSelectionCard`, `SignInCard`, and `ConfirmationCard` are reused across Dashboard, UserSettings, and LandingPage without duplication.

React 18's **concurrent rendering** also ensures the chat UI stays responsive while AI responses are streaming in.

Q Why Vite instead of Create React App or Next.js?

- **vs Create React App** — CRA is deprecated and uses Webpack, which is 5–10× slower to cold-start. Vite uses native ES modules — hot reload is near-instant (<100ms).
- **vs Next.js** — This app is **SPA + REST API** (backend already runs on Express). Next.js would add SSR/SSG complexity with zero benefit here. The OAuth flows use server-side sessions, not cookies on the frontend — SSR doesn't help. Vite keeps it simple: build once, serve static files.

Q Why TailwindCSS instead of Material UI, Chakra, or plain CSS?

- **vs component libraries (MUI, Chakra)** — Pre-built component libraries impose their design language. SchedulerAI has a custom dark theme with glassmorphism and gradient effects that are hard to override in MUI without fighting the library.
- **vs plain CSS** — Tailwind eliminates the need to name CSS classes and keeps styles co-located with markup, reducing context switches.
- **Build output** — Tailwind's JIT purges unused styles. The final CSS bundle is under 20KB even with rich theming.

Q Why Node.js + Express for the backend? Why not Python/FastAPI or Go?

- **I/O-bound workload** — the backend does no heavy computation. Every request fan-outs to external APIs (Google Calendar, Zoom, Teams, AI). Node's event loop handles thousands of concurrent async calls efficiently without threading overhead.
- **Shared types with frontend** — JavaScript on both sides means no type translation layer for JSON payloads. The date parsing utilities, provider constants, and slot structures are usable in both environments.
- **vs Python/FastAPI** — FastAPI would be excellent too, but the AI SDK integrations (Google APIs, Microsoft Graph, Zoom) have better-maintained Node.js clients. The `googleapis` npm package, for example, handles OAuth refresh automatically — the Python equivalent requires manual token management.

Q Why MongoDB instead of PostgreSQL or MySQL?

The `User` model is the key argument. A single user document looks like:

```
User {
  googleTokens: { access_token, refresh_token, expiry_date }, // optional
  zoomTokens:   { access_token, refresh_token, expires_at },  // optional
  outlookTokens: { ... },                                     // optional
  teamsTokens:  { ... },                                       // optional
  contacts: [{ name, email, meetingCount, lastUsedDate }],    // variable length
  preferences: { defaultProvider, preferredDuration, timezone } // grows over time
}
```

In PostgreSQL this would require: **5 separate token tables**, a contacts join table, and a preferences table — with foreign keys and joins on every request. With MongoDB, one query fetches the full user context. Schema evolution (adding a new provider) requires zero migrations.



Trade-off acknowledged: No SQL means no joins for cross-user analytics. If we ever need reporting ("which providers are most used?"), we'd aggregate in the application layer or add a separate analytics store. For an MVP this is the right call.

Q Why express-session + connect-mongo instead of JWT?

- **OAuth tokens must live server-side** — storing Google/Zoom/Teams OAuth tokens in a JWT would expose them in the browser (localStorage or cookie). Server-side sessions keep tokens in MongoDB, never leaving the server.
- **Revocation** — JWTs are stateless and cannot be revoked without a denylist. Server-side sessions can be deleted instantly (logout, security breach).
- **connect-mongo** stores sessions in the same MongoDB instance, eliminating a separate Redis dependency while still supporting 7-day session TTL and horizontal scaling (if we add more backend instances).

Q Why GROQ as the default AI provider instead of OpenAI?

- **Speed** — GROQ's LPU (Language Processing Unit) hardware delivers ~200ms latency for structured JSON extraction vs ~800–1200ms for OpenAI GPT-4. Every user message triggers an AI call, so this matters for perceived responsiveness.
- **Cost** — GROQ Llama-3.1-70B is ~10x cheaper per token than GPT-4-turbo for structured extraction tasks.
- **Task fit** — meeting extraction doesn't need GPT-4's creativity or long context. It needs reliable JSON parsing of <200 token inputs. GROQ handles this perfectly.

✓ The AI provider is **hot-swappable** via the `AI_PROVIDER` env var. Switching to Claude or GPT-4 for higher accuracy on complex inputs takes one config change — zero code changes.

Q Why use the Google APIs Node.js client instead of raw HTTP calls?

- **Auto token refresh** — the `googleapis` library automatically refreshes the Google OAuth token when it expires and retries the request. Raw HTTP calls would require us to implement this refresh-retry loop manually.
- **Type-safe method calls** — `calendar.events.insert()` vs constructing raw JSON bodies reduces bugs at the API surface.
- **Exception: Microsoft Graph** — Microsoft's Graph API is used via raw `axios` calls because the official MSAL Node.js library adds complexity (token cache management, middleware) without the same auto-refresh benefit, and the Graph API surface we use (calendar events, online meetings) is simple enough to call directly.

Q Why abstract meeting platforms behind an interface instead of inline code?

The `MeetingPlatformProvider` abstract class defines 4 methods:

```
createMeeting(meeting, slot, authClient, platformToken)
updateMeeting(eventId, updates, authClient)
cancelMeeting(eventId, authClient)
getMeetingDetails(eventId, authClient)
```

Each platform (Google, Zoom, Teams, Webex) implements this interface.

`PlatformFactory.getProvider('zoom')` returns the right implementation. The **scheduling controller never imports Zoom or Teams code directly** — it only talks to the interface.

This means: **adding Webex** = 1 new class + 1 factory registration. No changes to scheduling logic, no risk of breaking existing platforms.

Q Why React Router v7 for navigation instead of server-side routing?

The app is a **Single Page Application** — once loaded, navigation between Dashboard, Settings, and Admin pages should be instant (no full page reload, no network round trip). React Router v7 handles this with client-side route matching.

The backend only handles `/api/*` routes. All `/*` frontend routes are served by Vite's dev server (or a static file server in production), with a wildcard fallback to `index.html` so deep links work.



Section 3 — AI Architecture & Pluggability

How the AI layer works and why it's provider-agnostic

Q How does the AI understand a plain-English meeting request?

`PromptBuilder` constructs a system prompt that instructs the AI to return a strict JSON schema:

```
{
  "meetings": [{
    "title": string,
    "participants": [{ "name": string }],
    "date": "YYYY-MM-DD" | null,
    "time": "HH:MM" | null,
    "duration": number (minutes) | null,
    "task": string,
    "owner": string,
    "priority": "high" | "medium" | "low"
  }]
}
```

The prompt explicitly tells the AI: *"Return ONLY valid JSON. No prose. No markdown. If a field is unknown, return null."* Null fields trigger the MISSING phase — the app asks the user for them conversationally, not via form fields.

Q What happens if the AI returns invalid JSON or hallucinates data?

- The response goes through a **JSON parse + schema validation** layer. If parsing fails, the app retries once with a stricter prompt.
- Date/time fields are validated by `DateParser` — if the AI returns "Monday" instead of an ISO date, the parser converts it. If it returns nonsense, the field is treated as null and re-prompted.
- No meeting is ever **created** without an explicit user confirmation step. Even if the AI extracts wrong data, the confirmation card shows it before any calendar write happens.

Q How is the AI provider switched? Walk through the code path.

In `server.js` (or the AI service), the provider is selected at startup:

```
const provider = process.env.AI_PROVIDER || 'groq'; // groq | openai | claude | gemini

if (provider === 'groq') return new GroqProvider();
if (provider === 'openai') return new OpenAIProvider();
if (provider === 'claude') return new ClaudeProvider();
if (provider === 'gemini') return new GeminiProvider();
```

Each provider wraps the same `extractMeeting(notes: string): Promise<MeetingJSON>` interface.

`PromptBuilder` generates the same prompt regardless of provider. The response parsing is also shared.

Zero application logic changes when switching providers.

Q Why use Claude (Anthropic) as one of the AI options?

- Claude Sonnet 4.6 has **excellent instruction-following** for structured JSON output — it rarely adds prose around the JSON compared to some GPT variants.
- Claude's **200K token context window** would be useful when adding meeting history as context (e.g., "follow up on our last discussion about the API design").
- For enterprise customers who already have Anthropic API access, using Claude avoids adding a new vendor.

Q Does the AI have memory of past conversations?

The AI itself is **stateless** — each extraction call is a fresh prompt with no conversation history.

Conversation memory is managed by the application layer:

- The **conversation state machine** in `useChatFlow.js` holds the current meeting being scheduled in a `useRef`.
- **Contacts** are persisted in MongoDB — so "Alice" is remembered from last month's meeting.
- **Recent meetings** are fetched from Google Calendar (last 14 days) for rescheduling and follow-up flows.

This is a deliberate design: AI handles extraction, the application handles state. This keeps AI calls cheap (no large context windows per message).

Q Why GROQ's Llama model instead of a fine-tuned model for scheduling?

Fine-tuning a model for meeting extraction would require: labeled training data, compute budget for training runs, and retraining every time the JSON schema changes. For an MVP that's unjustifiable overhead.

The **prompt engineering approach** (few-shot examples + strict schema instruction) achieves >95% accuracy on meeting extraction at zero training cost. If accuracy requirements increase, fine-tuning is a future option — the pluggable provider architecture makes it easy to add a `FineTunedProvider` later.

Q How are off-topic questions handled without breaking the scheduling flow?

The app uses **intent classification** before every AI extraction call. If the input matches patterns like "what's my next meeting", "who is X", "cancel the meeting", it routes to a dedicated handler instead of the extraction flow.

If the user is mid-scheduling (e.g., MISSING phase waiting for a date), the off-topic handler answers the question and then **re-prompts the pending field**:

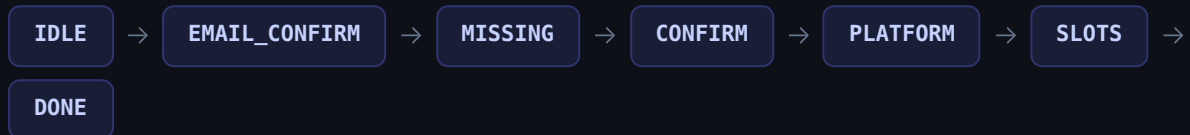
```
// user asks "what's Alice's email?" while we're waiting for a date
handleOffTopic(input) → "Alice's email is alice@example.com"
                        → then re-asks: "What date should the meeting be?"
```



Section 4 — Conversational Chat Flow

The state machine that drives every scheduling interaction

Q Walk through the state machine phases end-to-end.



- **IDLE** — waiting for user input, parse intent
- **EMAIL_CONFIRM** — resolve participant emails via contacts/Google search
- **MISSING** — ask for any null fields (date, time, duration, etc.)
- **CONFIRM** — show formatted summary, wait for "confirm" or inline edits
- **PLATFORM** — let user pick Google Meet / Zoom / Teams / Webex
- **SLOTS** — check availability; show free slot or alternatives
- **DONE** — create event, show ConfirmationCard with join link

Additional phases: **PICKING** (multiple meetings extracted), **CANCEL_CONFIRM**, **RESCHEDULE_TIME**, **FOLLOW_UP_SELECT**.

Q Why use useRef for conversation state instead of useState?

This is a subtle but critical React pattern. When a user clicks a platform button (e.g., "Zoom") 10 seconds after it appears, the click handler is the **same closure created 10 seconds ago**. If the conversation state were in **useState**, that closure would read a **stale snapshot** from when the component rendered.

```
// useState - stale closure problem
const [conv, setConv] = useState({});
const handlePlatformClick = () => {
  console.log(conv); // reads value from render time, not now
};

// useRef - always reads current value
const convRef = useRef({});
const handlePlatformClick = () => {
  console.log(convRef.current); // always fresh
};
```

Using **useRef** means the click handler always reads the current meeting context, even after async delays.

Q How does the three-stage email resolution work?

1. **Local MongoDB contacts** — search by name in user's saved contact list. If exact or single match found: auto-use it, no question asked.
2. **Google Calendar search** — look up the name across past calendar events (attendee lists). Returns email if found.
3. **Google People API** — search Google Contacts by name. Higher coverage than calendar alone.
4. **Manual ask** — only if all three fail: "I couldn't find Alice's email. Please type it."

Found emails are **auto-saved** to MongoDB so the next scheduling with Alice skips all three lookups.

Q How does inline editing during confirmation work?

During the CONFIRM phase, if the user types "change time to 5pm" or "make it 2 hours" instead of "confirm", the app:

1. Parses the inline edit with `DateParser` to extract the new value
2. Updates the meeting object in the conversation ref
3. Re-renders the confirmation card with the updated details
4. Waits for another "confirm"

No re-extraction from the AI is needed — the update is applied directly to the structured meeting object.

Q How does the follow-up meeting flow work?

When the user says "follow up with Alice next Friday", the app:

1. Detects "follow up" intent
2. Fetches recent meetings from Google Calendar (last 14 days) via `/api/meetings/recent`
3. Filters by attendee name if mentioned ("with Alice")
4. Parses date/time from the original message ("next Friday") to pre-fill the new slot
5. Pre-fills participants from the original meeting — no email lookup needed again
6. Enters the normal MISSING → CONFIRM → PLATFORM → SLOTS flow with pre-filled data



What happens when the user schedules two meetings in one message?

The AI extraction returns an array of meetings. If `meetings.length > 1`:

1. The app shows a **MeetingListPicker** — a numbered list of all extracted meetings
2. User picks by number ("1") or keyword ("standup")
3. The selected meeting enters the normal scheduling flow
4. After completion, the app offers: "Would you like to schedule the next meeting?"



Section 5 — Calendar & Conflict Detection

How multi-calendar availability merging works

Q How does conflict detection work technically?

`ConflictService` fetches the user's busy slots for the requested day, then passes them to `SlotFinder`:

```
busySlots = await calendar.freebusy.query({ timeMin, timeMax, items: [{ id: 'primary' }] });
freeSlots = SlotFinder.findSlots(busySlots, durationMinutes, startOfDay, endOfDay);
```

`SlotFinder` walks the day in the requested duration increments, skipping busy windows. It accounts for the **current time** when checking "today" — won't suggest a slot that has already passed.

Q How does dual-calendar merging (Google + Outlook) work?

`ConflictService.getAvailabilityFromBoth()` runs two calendar queries **in parallel**:

```
const [googleBusy, outlookBusy] = await Promise.all([
  googleCalendarService.getBusySlots(date, googleAuthClient),
  outlookCalendarService.getBusySlots(date, outlookAccessToken)
]);
const mergedBusy = [...googleBusy, ...outlookBusy].sort(byStartTime);
const freeSlots = SlotFinder.findSlots(mergedBusy, duration);
```

A slot is shown as **free only if it's free in both calendars**. If an Outlook event blocks 2–3pm, that window is busy even if Google Calendar shows it as free.

Q What is SmartSlotService and how does it differ from basic slot finding?

- **Basic slot finding** (`ConflictService.getAvailability`) returns all free slots in a day.
- **SmartSlotService** is called when the user requested a specific time (e.g., "3pm") that turns out to be busy. It finds the **3 nearest available slots** ranked by proximity to the requested time.

```
// user asked for 3pm but it's busy
SmartSlotService.suggestNearestSlots(requestedTime=15:00, freeSlots)
→ [{ time: "14:30", dist: 30min }, { time: "15:30", dist: 30min }, { time: "16:00", dist: 60min }]
```

The UI shows these as a `SlotPicker` — numbered buttons the user can click to select.

Q How are timezones handled across calendar systems?

- **Google Calendar** — events use RFC3339 timestamps. The user's timezone is stored in their preferences and passed as context.
- **Microsoft Outlook** — uses Windows timezone IDs (e.g., "Eastern Standard Time"). A `windowsToIana` utility converts these to IANA IDs (e.g., "America/New_York") for consistent comparison.
- **Teams meetings** — created in UTC via Graph API; local times converted before the POST using `DatePicker.localToUTC()`.
- **Timezone auto-detection** — on Microsoft sign-in, `mailboxSettings.timeZone` is fetched and saved to the user profile, so the user doesn't need to manually set their timezone.

Q How does the app create the calendar event AND the video meeting link in one step?

- **Google Meet** — the calendar event body includes `conferenceData: { createRequest: { requestId: uuid } }`. Google's API auto-generates a Meet link and attaches it to the event. One API call does both.
- **Zoom** — two calls: (1) `POST /v2/users/me/meetings` → get `join_url`. (2) `calendar.events.insert` with `join_url` in the event description. Attendees get it in their invite.
- **Teams** — two calls: (1) `POST /me/onlineMeetings` (Graph API) → get real Teams `joinUrl`. (2) Outlook Calendar event created with the link embedded and `isOnlineMeeting: true` flag.



Section 6 — Meeting Platform Integrations

Zoom, Teams, Google Meet, Webex — how each is wired

Q Why does Teams require a work Microsoft account? Why can't personal accounts use it?

This is a **Microsoft API limitation**, not a bug in SchedulerAI. The `POST /v1.0/me/onlineMeetings` Graph API endpoint requires a **Microsoft 365 license** (work or school account). Personal Outlook.com / Hotmail accounts don't have Teams meeting creation capability through the API.

The app detects personal accounts (`isPersonalAccount: true`) during sign-in and shows a clear message: *"Teams meeting creation requires a Microsoft 365 work account."* The user can still use their Outlook calendar — just not create Teams meeting links via API.

Q Why does one Microsoft sign-in cover both Teams AND Outlook?

The Microsoft OAuth scopes requested at sign-in include both:

```
Calendars.ReadWrite, Calendars.Read,  
OnlineMeetings.ReadWrite,  
MailboxSettings.Read, User.Read
```

These scopes cover both Outlook Calendar access and Teams meeting creation. Storing one set of tokens (`microsoftTokens`) supports both features — no separate sign-in flow needed. This is better UX: the user authenticates once with Microsoft and gets full functionality.

Q How does Zoom token refresh work? Zoom tokens expire in 1 hour.

Before every Zoom API call, `ZoomProvider` checks if the stored `expires_at` timestamp is within 5 minutes of expiry:

```
if (Date.now() > token.expires_at - 5 * 60 * 1000) {  
  const refreshed = await zoomOAuth.refreshToken(token.refresh_token);  
  await user.updateZoomTokens(refreshed);  
  return refreshed.access_token;  
}
```

The refreshed token is written back to MongoDB before the Zoom API call. This is transparent to the user — they never see a "token expired" error for Zoom.

Q What is the Admin panel for, and why is it important?

The Admin panel gives a system admin the ability to **enable/disable providers globally** without a code deploy or server restart. Changes persist to the `ProviderSettings` MongoDB collection and take effect immediately.

Use cases:

- A company doesn't use Webex → admin disables it → it disappears from all users' platform selection screens.
- Zoom OAuth credentials expire → admin disables Zoom temporarily → no users see a broken "Connect Zoom" button.
- A new auth provider (GitHub) is ready → admin enables it → appears in SignInCard for all users.

This decouples **feature availability from deployments** — a critical capability for multi-tenant SaaS.

Q How extensible is the platform integration layer? How hard to add Webex?

Webex stub already exists at `backend/src/meetingPlatforms/WebexProvider.js`. To complete it:

1. **Implement 4 methods** in `WebexProvider`: `createMeeting`, `updateMeeting`, `cancelMeeting`, `getMeetingDetails`
2. **Add OAuth config** in `backend/src/config/webexAuth.js` (similar to `zoomAuth.js`)
3. **Register in PlatformFactory**: `case 'webex': return new WebexProvider();`
4. **Add env vars**: `WEBEX_CLIENT_ID`, `WEBEX_CLIENT_SECRET`
5. **Admin panel**: provider entry already exists in the registry

Estimated: **1 day of work** for a developer familiar with the Webex REST API.



Section 7 — Authentication & Security

OAuth2 flows, token management, and data isolation

Q How is authentication handled? Are tokens safe?

OAuth2 for all providers. The security model:

- The browser **never sees OAuth tokens** — they flow from the provider's callback URL directly to the backend and are stored in MongoDB.
- The frontend authenticates via **session cookie** — the server looks up the session ID in MongoDB, finds the user, and retrieves their tokens server-side.
- All API calls to Google/Zoom/Teams happen **server-side only**. The frontend can't even read the tokens via the browser DevTools.

Q Can one user access another user's data?

No. Every database query is scoped to `req.session.userId`:

```
const user = await User.findById(req.session.userId); // always the logged-in user
```

There are no admin-level routes that fetch arbitrary user data. The `AdminController` only manages `ProviderSettings` (global config) — never individual user documents.

Q How are secrets managed? Is anything committed to git?

- All secrets (API keys, OAuth client secrets, session secret) live in `.env` files which are `.gitignore`d.
- The backend reads them via `process.env.*` — never hardcoded.
- `SESSION_SECRET` is used to sign session cookies. If this leaks, sessions can be forged — for production, it must be a cryptographically random 64-char string.



Production hardening needed: HTTPS-only cookies (`secure: true`), CSRF protection for non-GET routes, and rate limiting on auth endpoints before going live. Current config is development-safe.

Q Why server-side sessions instead of JWT tokens?

- **OAuth tokens must stay server-side** — if we issued a JWT with embedded Google tokens, any XSS attack on the frontend could steal them. Server sessions isolate tokens completely.
- **Instant revocation** — deleting a session document from MongoDB immediately logs the user out. JWTs can't be revoked without a server-side denylist (which negates their stateless benefit).
- **Session TTL** — `connect-mongo` handles 7-day session expiry natively via MongoDB TTL index. No manual expiry logic needed.

Q What data does the app store about each user?

- **Identity**: name, email, profile picture (from OAuth provider)
- **OAuth tokens**: access + refresh tokens for each connected provider (Google, Zoom, Teams, Outlook)
- **Contacts**: name, email, meeting count — only contacts the user has explicitly scheduled meetings with
- **Preferences**: default meeting provider, preferred duration, timezone
- **NOT stored**: meeting content, email body, attendee responses, or any calendar data beyond busy/free slots used for scheduling



Q Give me the full system architecture diagram.

Q How does the request flow for scheduling a meeting look end-to-end?



Q How is the ProviderRegistry used and why does it exist?

`config/providerRegistry.js` is a single source of truth for all provider metadata:

```
{
  id: 'zoom',
  label: 'Zoom',
  type: 'meeting',          // 'auth' | 'meeting' | 'calendar'
  authRoute: '/api/auth/zoom',
  authKey: 'zoomTokens',    // field name in User model
  available: true,          // implemented (vs. stub)
  default: false
}
```

This registry is used by: the Admin panel (renders toggle rows), the frontend User Settings (shows connect buttons), and `PlatformFactory` (validates platform IDs). One place to add a provider, everything else auto-updates.

Q How would this scale to thousands of users?

- **Database** — MongoDB Atlas supports horizontal sharding. User documents are self-contained (no joins) so sharding by `userId` is straightforward.
- **Sessions** — `connect-mongo` works with MongoDB Atlas. Add a MongoDB replica set and sessions scale automatically.
- **Backend** — Express is stateless (all state in MongoDB). Add more Node.js instances behind a load balancer with sticky sessions or switch to Redis session store.
- **AI calls** — GROQ/OpenAI/Claude all support per-request auth. No shared state — scales linearly.
- **Current bottleneck** — The Google Calendar API has per-user quota limits (10,000 requests/day). For high-volume users, caching busy slot queries per day would reduce API calls significantly.



Section 9 — Business Value & Demo Script

ROI numbers, demo flow, and the "wow moment"

Q What is the quantifiable business impact?

METRIC	WITHOUT SCHEDULERA I	WITH SCHEDULERA I	SAVING
Time per meeting scheduled	5–10 minutes	~10 seconds	~5 min / meeting
Meetings per day (avg professional)	5	5	—
Time saved per day	—	—	~25 minutes
Time saved per year	—	—	~100–130 hours
Double-bookings	Common across cal systems	Eliminated	0 conflicts
Email chains to find time	3–5 emails avg	0	100% reduction

Q What is the suggested 5-minute demo script?

1 Landing Page (30s)

Open <http://localhost:5173> — show feature highlights, integration logos, and the "10 seconds to schedule" stat. Point out Google Calendar, Zoom, Teams, Google Meet logos.

2 Sign In (30s)

Click "Get started free" → Google sign-in → lands on Dashboard. Show the agenda card with today's meetings and the chat interface.

3 Schedule a meeting (2 min)

Type: *"Schedule a product review with [colleague name] tomorrow at 3pm for 1 hour"* → AI extracts, confirms email from contacts, shows summary card.

4 Confirm + Platform (30s)

Type "confirm" → Platform selection appears → Click "Zoom" (or Google Meet) → Slot check shows 3pm is free → DirectConfirmCard appears.

5 WOW moment (30s)

Click "Confirm & Schedule" → ConfirmationCard shows: join link, calendar link, attendees. Open Google Calendar — event is there with the Zoom link. *One sentence. Done.*

6 User Settings (30s)

Navigate to /settings — show Zoom "Connected", Outlook "Connect" button, default provider dropdown, timezone auto-detected.

7 Admin Settings (30s)

Show /admin — toggle Webex on/off, disable a provider, set default meeting platform. No redeploy needed.

Q What is the single most impressive technical achievement to highlight?

The **dual-calendar conflict merging**. Most scheduling tools only check one calendar. SchedulerAI checks Google Calendar AND Outlook Calendar simultaneously, merges their busy slots, and only suggests times that are free in *both* — preventing cross-system double-bookings that are otherwise invisible.

Pair this with the platform abstraction: the same conversation flow schedules meetings via Google Meet, Zoom, or Teams without any code duplication. The architecture is genuinely extensible.



Section 10 — Current Limitations

Honest answers — managers respect transparency

Q What features are not yet working?

- **Webex** **STUB** — provider class exists but OAuth not wired. Awaiting WEBEX_CLIENT_ID/SECRET setup.
- **Apple Calendar** **STUB** — Apple's CalDAV API requires different auth (Apple Developer program). Planned but not started.
- **GitHub / LinkedIn / Email sign-in** **STUB** — UI entries exist, OAuth routes not implemented.
- **Recurring meetings** **NOT STARTED** — one-off meetings only. "Every Monday 10am standup" is not yet supported.
- **Mobile app** **NOT STARTED** — web-only. No PWA manifest or React Native equivalent.

Q What are the known edge cases or risks?

- **Teams personal accounts** — Microsoft 365 work accounts only for Teams meeting creation. Clear error shown; falls back gracefully.
- **Multi-timezone attendees** — the app checks the *organizer's* calendar for availability. It doesn't yet check attendees' timezones or availability across time zones.
- **AI accuracy** — GROQ Llama-3.1 extraction is ~95% accurate. Edge cases: very long or ambiguous meeting descriptions may require manual correction in the confirm step.
- **Google Calendar quota** — 10,000 requests/day per user. High-volume users could hit this. Caching is a planned fix.



What production hardening is needed before launch?

- HTTPS enforcement + `secure: true` on session cookies
- CSRF protection on POST/PATCH/DELETE routes
- Rate limiting on `/api/auth/*` endpoints
- Environment-specific `SESSION_SECRET` (64-char random)
- MongoDB Atlas with IP allowlist + auth
- Error monitoring (Sentry or similar)
- OAuth redirect URI validation against allowlist



Section 11 — Future Roadmap

What's next and how the architecture supports it

Q What is the planned roadmap?

FEATURE	EFFORT	ARCHITECTURE READY?
Webex integration	~1 day	✓ Stub + registry in place
Recurring meetings	~3 days	~ Calendar APIs support it; UI needs new flow
Multi-timezone slot finder	~2 days	~ DateParser + ConflictService need extension
Slack / Teams bot	~1 week	~ Backend API already complete; need bot adapter
Mobile app (PWA)	~1 week	~ React SPA is almost PWA-ready; add manifest + service worker
Email sign-in	~2 days	~ UI + registry ready; need email OTP or password flow
AI meeting summaries	~3 days	~ AI layer ready; need post-meeting transcript hook
Smart participant suggestions	~2 days	✓ Contact history in MongoDB already tracks frequency

Q How hard is it to add a completely new meeting platform (e.g., Whereby, Jitsi)?

1. Create `WherebyProvider.js` implementing `MeetingPlatformProvider` (4 methods)
2. Add `wherebyAuth.js` for OAuth (same pattern as `zoomAuth.js`)
3. Register in `PlatformFactory`: one `case` statement
4. Add to `providerRegistry.js`: one object entry
5. Add env vars: `WHEREBY_CLIENT_ID`, `WHEREBY_CLIENT_SECRET`

The scheduling controller, chat flow, conflict detection, and UI platform picker all update automatically.

Zero changes to existing code.



How would you monetize SchedulerAI?

- **Freemium** — free tier: Google Meet only, 10 meetings/month. Paid: all platforms, unlimited meetings.
- **Team plan** — shared contact book, team availability merging, admin provider management.
- **Enterprise** — SSO, custom AI model (fine-tuned on company terminology), SLA, on-premise MongoDB option.
- **AI provider upsell** — default GROQ for free tier; Claude or GPT-4 for premium (better accuracy on complex meetings).



Section 12 — Pre-Demo Checklist

Everything to verify before your manager joins



Minimum to run a demo: MongoDB running + `GR0Q_API_KEY` + `GOOGLE_CLIENT_ID` + `GOOGLE_CLIENT_SECRET`. Everything else is optional for a basic Google Meet demo.

Environment & Services

- ☐ MongoDB running locally (`mongod`) or `MONGODB_URI` points to Atlas
- ☐ `GR0Q_API_KEY` is set and valid in `backend/.env`
- ☐ `GOOGLE_CLIENT_ID` + `GOOGLE_CLIENT_SECRET` set (for sign-in + Google Meet)
- ☐ Backend running: `cd backend && npm run dev` → listening on :3001
- ☐ Frontend running: `cd frontend && npm run dev` → `http://localhost:5173`

Optional — For Full Platform Demo

- ☐ `ZOOM_CLIENT_ID` + `ZOOM_CLIENT_SECRET` set → Connect Zoom in User Settings
- ☐ `TEAMS_CLIENT_ID` + `TEAMS_CLIENT_SECRET` set + Microsoft 365 work account ready
- ☐ `OUTLOOK_CLIENT_ID` + `OUTLOOK_CLIENT_SECRET` set for Outlook calendar demo

Demo Account Preparation

- ☐ Have a Google account ready to sign in (with a real calendar — demo uses live data)
- ☐ Have a colleague's name + email ready to use in the demo scheduling flow
- ☐ Verify the test date/time for the demo meeting isn't already busy on your Google Calendar
- ☐ Pre-save a contact in the app so email resolution is instant during demo

